Autonomous Data Analyse Agent - Project Architecture

Executive Summary

The Autonomous Data Analyse Agent is an interactive AI-driven data science application. It features a modern Python architecture where the frontend (User Interface) is built with Streamlit and the backend (Core Engine) is neatly organized within the `src` directory, containing modules for data ingestion, validation, LLM orchestration, and secure code execution.

1. The Frontend (User Interface)

Location: Root directory

- **`app.py`**: This is the main entry point for the application. Running `streamlit run app.py` launches the UI. It acts as the bridge connecting the user to the Backend, handling file uploads, user prompts, chat history, and dynamic rendering of charts/code generated by the agent.

2. The Backend (Core Engine & Logic)

Location: `src/autonomous_data_analyse_agent/`

All the underlying intelligence, code execution, and data processing is neatly organized here.

2.1 `core/` (The Brains)

This folder handles the AI logic and code execution.

- **`agent.py`**: Contains the `AutonomousDataAgent` class. This handles talking to LLMs (OpenAI, Groq, Google, Ollama), manages the conversation history, and holds the System Prompts that tell the AI how to behave (e.g., instructing it to manipulate data securely).
- **`sandbox.py`**: Contains the `LocalExecutionSandbox`. When the AI writes Python code, this file intercepts that code and executes it in an isolated environment so it can modify your DataFrame securely without breaking the app.

2.2 `data/` (Data Processing)

This folder handles everything related to reading and understanding the datasets you upload.

- **`ingest.py`**: Loads raw CSV, Excel, or JSON files into Pandas DataFrames.
- **`detect.py`**: Auto-detects data types and properties from the raw data.
- **`profile.py`**: Analyzes the uploaded data to generate the text summary (missing values, column types) that is fed to the AI.
- **`validate.py`**: Audits the dataset for quality issues (like empty columns or zero variance).

2.3 `ml/` (Machine Learning Tools)

A suite of helper functions that the AI can call upon when it decides to build predictive models.

- **`pipeline.py`**: Helpers for preprocessing, such as `train_test_split`.
- **`model.py`**: Logic for training predictive models.
- **`evaluate.py`**: Logic for evaluating model performance (e.g., cross-validation).

2.4 `utils/` (Utilities)

- General helper functions and generic utilities used across the backend.

3. Testing & Experimentation

You have a mix of test scripts to verify the agent works outside of the UI:

- **`tests/test_full_pipeline.py`**: A formal test file for the entire data and agent pipeline.
- **`test_phase1.py`, `test_phase2.py`, `test_phase5.py`, `all_test.py`**: Scratch scripts in the root directory used to build and test specific backend components step-by-step before wiring them up to `app.py`.

4. Configuration & Dependencies

- **`.env`**: Stores environment variables and API keys (Google, OpenAI, Groq) securely.
- **`pyproject.toml`**, **`uv.lock`**, **`requirement.txt`**: Dependency management files declaring which Python packages (like pandas, streamlit, langchain, etc.) the project needs to run.

5. Execution Flow

1. **Upload**: User uploads data via **`app.py`** (Frontend).
2. **Process**: Data is parsed and summarized by the tools in **`src/.../data/`**.
3. **Prompt**: User types a request, which **`app.py`** sends to **`agent.py`** (The LLM).
4. **Code Generation**: The LLM writes Python code, which is handed to **`sandbox.py`** to be executed.
5. **Execution**: **`sandbox.py`** runs the code, saves any charts, and returns the output/errors back to **`app.py`** to display to the user.